



Өгөгдөл хувиргах dplyr

Data transformation cheat sheet – хүснэгтийн мөр, баганыг шүүх, эрэмбэлэх, нэгтгэх, холбох

dplyr нь өгөгдөл боловсруулах нийтлэг асуудлыг шийдэх, нэгэн хэвийн «үйл үг» функцуудын цуглуулга юм. Функц бүр эхний аргументаар дата фрейм авч, шинэ дата фрейм буцаадаг тул `pipe` `|>`-ээр гинжлэн холбоно: `x |> f(y)` нь `f(x, y)`-тэй адил.

Tidy өгөгдөл

dplyr нь **tidy** өгөгдөлтэй ажиллана: хувьсагч бүр өөрийн **баганад**, ажиглалт (тохиолдол) бүр өөрийн **мөрөнд** байрлана. Ингэснээр векторжуулсан үйлдлүүд багана бүр дээр шууд ажиллана.

```
library(dplyr)
starwars |> filter(species == "Droid")
```

Жишээнүүдэд dplyr-т багтсан `mtcars`, `starwars`, `storms` өгөгдлийг ашиглав.

Тохиолдлыг хураангуйлах

Хураангуй функцууд вектор авч, **нэг утга** буцаана.

summarise(.data, ...) Хураангуй статистикийн хүснэгт үүсгэнэ. Мөн `summarize()`.

```
mtcars |> summarise(avg = mean(mpg))
```

count(.data, ..., wt = NULL, sort = FALSE, name = NULL) Нэг буюу хэд хэдэн хувьсагчийн утга бүрийн мөрийн тоог тоолно. Мөн `tally()`, `add_count()`, `add_tally()`.

```
mtcars |> count(cyl)
```

Тохиолдлыг бүлэглэх

`group_by(.data, ..., .add = FALSE, .drop = TRUE)` нь хүснэгтийн «бүлэглэсэн» хувилбарыг үүсгэнэ. Дараагийн dplyr функцууд бүлэг тус бүр дээр тусад нь ажиллаж, үр дүнг нэгтгэнэ.

```
mtcars |>
  group_by(cyl) |>
  summarise(avg = mean(mpg))
```

ungroup(x, ...) Бүлэглэлийг арилгана.

rowwise(.data, ...) Мөр бүрийг тусдаа бүлэг болгоно. Жагсаалт-баганатай ажиллахад тохиромжтой.

```
starwars |>
  rowwise() |>
  mutate(film_count = length(films))
```

Нэг удаагийн бүлэглэл: .by

`summarise()`, `mutate()`, `filter()`, `slice()` зэрэгт `.by` = аргументаар тухайн үйлдлийн хүрээнд л бүлэглэнэ – дараа нь `ungroup()` хийх шаардлагагүй.

```
mtcars |> summarise(avg = mean(mpg), .by = cyl)
```

Тохиолдолтой ажиллах

Мөр сонгох

Логик нөхцөл эсвэл байрлалаар мөрүүдийг сонгож, шинэ хүснэгт буцаана.

filter(.data, ..., .preserve = FALSE) Логик нөхцөлийг хангасан мөрүүдийг сонгоно.

```
mtcars |> filter(mpg > 20)
```

distinct(.data, ..., .keep_all = FALSE) Давхардсан мөрүүдийг хасна.

```
mtcars |> distinct(gear)
```

slice(.data, ...) Мөрүүдийг байрлалаар нь сонгоно.

```
mtcars |> slice(10:15)
```

slice_sample(.data, ..., n, prop, weight_by = NULL, replace = FALSE) Санамсаргүйгээр мөр сонгоно. `n` – мөрийн тоо, `prop` – эзлэх хувь.

```
mtcars |> slice_sample(n = 5, replace = TRUE)
```

slice_min(.data, order_by, ..., n, prop, with_ties = TRUE) Хамгийн бага утгатай мөрүүд. Мөн `slice_max()`.

```
mtcars |> slice_min(mpg, prop = 0.25)
```

slice_head(.data, ..., n, prop) Эхний мөрүүд. Мөн `slice_tail()`.

```
mtcars |> slice_head(n = 5)
```

Логик ба булийн операторууд

<code>==</code>	Тэнцүү	<code>!=</code>	Тэнцүү биш
<code><</code>	Бага	<code><=</code>	Бага буюу тэнцүү
<code>></code>	Их	<code>>=</code>	Их буюу тэнцүү
<code>is.na()</code>	Дутуу	<code>!is.na()</code>	Дутуу биш
<code>%in%</code>	Олонлогт багтах	<code>xor()</code>	Зөвхөн нэг нь
<code>&</code>	Ба	<code> </code>	Эсвэл
<code>!</code>	Үгүй	<code>between()</code>	Мужид багтах

`filter()` дотор таслалаар тусгаарласан нөхцөлүүд &-ээр холбогдоно: `filter(x > 1, y < 5)`. Дэлгэрэнгүйг ? `base::Logic`, ? `Comparison`-оос.

Мөрийг эрэмбэлэх

arrange(.data, ..., .by_group = FALSE) Нэг буюу хэд хэдэн баганын утгаар мөрүүдийг өсөхөөр эрэмбэлнэ. Буурахаар эрэмбэлэх бол `desc()`-тэй хамт ашиглана.

```
mtcars |> arrange(mpg)
mtcars |> arrange(desc(mpg))
```

Мөр нэмэх

add_row(.data, ..., .before = NULL, .after = NULL) Хүснэгтэд нэг буюу хэд хэдэн мөр нэмнэ (`tibble` багц).

```
cars |> add_row(speed = 1, dist = 1)
```

Хувьсагчтай ажиллах

Багана сонгох

Баганын утгыг вектороор эсвэл хүснэгтээр гаргаж авна.

pull(.data, var = -1, name = NULL, ...) Баганын утгыг нэрээр эсвэл байрлалаар вектор болгон авна.

```
mtcars |> pull(wt)
```

select(.data, ...) Баганыг шинэ хүснэгт болгон сонгоно.

```
mtcars |> select(mpg, wt)
```

relocate(.data, ..., .before = NULL, .after = NULL) Баганын байрлалыг өөрчилнө.

```
mtcars |> relocate(mpg, cyl, .after =
  last_col())
```

Сонголтын туслах функц

`select()`, `across()`, `relocate()`, `pivot_longer()` зэргт ашиглана.

```
mtcars |> select(mpg:cyl)
```

<code>contains(match)</code>	Нэрэнд заасан текст агуулсан
<code>starts_with(match)</code>	Нэр нь ...-ээр эхэлсэн
<code>ends_with(match)</code>	Нэр нь ...-ээр төгссөн
<code>matches(match)</code>	Регуляр илэрхийлэлд таарсан
<code>num_range(prefix, range)</code>	x1, x2, ... гэх мэт
<code>all_of(x) / any_of(x)</code>	Тэмдэгт векторт буй нэрс
<code>where(fn)</code>	Функц TRUE буцаасан (жишээ нь, <code>where(is.numeric)</code>)
<code>everything()</code>	Бүх багана
<code>last_col(offset)</code>	Сүүлийн багана
<code>:</code>	Мужаар, жишээ нь <code>mpg:cyl</code>
<code>!</code>	Үгүйсгэх, жишээ нь <code>!gear</code>
<code>&, </code>	Огтлолцол, нэгдэл

Олон баганад нэг функц хэрэглэх

Олон баганад нэг ба түүнээс дээш функцийг `summarise()` эсвэл `mutate()` дотор хэрэглэнэ.

across(.cols, .fns, ..., .names = NULL) Олон баганыг хураангуйлах буюу өөрчлөх.

```
mtcars |> summarise(across(everything(), mean))
mtcars |> mutate(across(where(is.numeric),
  round))
```

c_across(.cols) Мөр бүрээр олон баганыг тооцоолно.

```
df |> rowwise() |>
  mutate(x_total = sum(c_across(1:2)))
```

pick(...) Сонгосон баганыг хүснэгт болгон функцэд дамжуулна.

Шинэ хувьсагч үүсгэх

Векторжуулсан функцийг баганад хэрэглэнэ. Векторжуулсан функц нь вектор авч, **ижил урттай** вектор буцаана (ард талд нь жагсаав).

mutate(.data, ..., .keep = "all", .before = NULL, .after = NULL) Шинэ багана нэмэх буюу байгааг өөрчилнө. `.keep = "none"` (хуучнаар `transmute()`) – зөвхөн шинэ баганыг үлдээнэ.

```
mtcars |> mutate(gpm = 1 / mpg)
mtcars |> mutate(gpm = 1 / mpg, .keep = "none")
```

rename(.data, ...) Баганын нэрийг солино: `new = old`. Функцээр солих бол `rename_with()`.

```
cars |> rename(distance = dist)
cars |> rename_with(toupper)
```

Бүрэн жишээ

```
starwars |>
  filter(!is.na(mass), !is.na(height)) |>
  mutate(bmi = mass / (height / 100)^2) |>
  summarise(
    n = n(),
    avg_bmi = mean(bmi),
    max_bmi = max(bmi),
    .by = species
  ) |>
  filter(n > 1) |>
  arrange(desc(avg_bmi))
```

Дутуу утгыг хасч → шинэ хувьсагч үүсгэж → зүйл (species) тус бүрээр хураангуйлж → нэгээс олон төлөөлөгчтэйг үлдээж → эрэмбэлнэ.



Векторжуулсан функцууд

`mutate()` доторх векторжуулсан функцууд вектор авч, ижил урттай вектор буцаана.

Шилжүүлэх (offset)

`lag(x, n = 1)` Утгыг 1 байрлалаар хойшлуулна (өмнөх утга).

`lead(x, n = 1)` Утгыг 1 байрлалаар урагшлуулна (дараагийн утга).

Хуримтлагдсан нэгтгэл

<code>cumall()</code>	Хуримтлагдсан <code>all()</code>
<code>cumany()</code>	Хуримтлагдсан <code>any()</code>
<code>cummax()</code> / <code>cummin()</code>	Хуримтлагдсан их / бага утга
<code>cummean()</code>	Хуримтлагдсан дундаж
<code>cumprod()</code> / <code>cumsum()</code>	Хуримтлагдсан үржвэр / нийлбэр

Эрэмбэ (ranking)

<code>row_number()</code>	Дугаарлалт, тэнцүү утгад эхэлсэн нь түрүүлнэ
<code>min_rank()</code>	Тэнцүү утгад ижил эрэмбэ, дараагийнх нь үсэрснэ (1, 2, 2, 4)
<code>dense_rank()</code>	Тэнцүү утгад ижил эрэмбэ, завсаргүй (1, 2, 2, 3)
<code>percent_rank()</code>	<code>min_rank</code> -ийг [0, 1] мужид шилжүүлсэн
<code>cume_dist()</code>	Хуримтлагдсан тархалт
<code>ntile(x, n)</code>	<code>n</code> тэнцүү бүлэгт хуваана
<code>consecutive_id()</code>	Дараалсан ижил утгуудыг нэг бүлэг болгож дугаарлана

Математик

<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>^</code> <code>%/%</code> <code>%%</code>	Арифметик үйлдэл (<code>%/%</code> – бүхэл хуваалт, <code>%%</code> – үлдэгдэл)
<code>log()</code> <code>log2()</code> <code>log10()</code>	Логарифм
<code><=</code> <code><</code> <code>></code> <code>>=</code> <code>!=</code> <code>==</code>	Логик харьцуулалт
<code>between(x, left, right)</code>	<code>x >= left & x <= right</code>
<code>near(x, y)</code>	Бутархай тоонд найдвартай <code>==</code>

Бусад

`case_when(...)` Олон нөхцөлт `if_else()`. Эхний үнэн нөхцөлийн утгыг авна.

```
starwars |>
  mutate(type = case_when(
    height > 200 | mass > 200 ~ "аварга",
    species == "Droid"       ~ "дроид",
    .default                 = "бусад"
  ))
```

`case_match(x, ...)` Утгыг нөгөө утгаар солих `switch()`-ийн векторжуулсан хувилбар.

`case_match(x, "a" ~ 1, c("b", "c") ~ 2, .default = 0)`

`if_else(condition, true, false, missing = NULL)` Төрлийг чанд шалгадаг `ifelse()`.

`coalesce(...)` Векторуудаас дутуу биш эхний утгыг сонгоно.

`na_if(x, y)` Тодорхой утгыг `NA` болгоно.

`pmax()` / `pmin()` Элемент бүрээр их / бага утга.

`replace_na(x, 0)` `NA`-г утгаар солино (`tidyr`).

Хураангуй функцууд

`summarise()` доторх хураангуй функцууд вектор авч, нэг утга буцаана.

Тоолох

<code>n()</code>	Мөрийн тоо
<code>n_distinct()</code>	Давхардаагүй утгын тоо
<code>sum(!is.na())</code>	Дутуу биш утгын тоо

Байршил

<code>mean()</code>	Дундаж. Мөн логик векторт <code>mean(!is.na())</code> – хувь
<code>median()</code>	Медиан

Логик

<code>mean()</code>	<code>TRUE</code> -ийн эзлэх хувь
<code>sum()</code>	<code>TRUE</code> -ийн тоо

Байрлал, дараалал

<code>first()</code>	Эхний утга
<code>last()</code>	Сүүлийн утга
<code>nth(x, n)</code>	<code>n</code> дэх утга

Эрэмбэ

<code>quantile()</code>	<code>n</code> -р квантиль
<code>min()</code> / <code>max()</code>	Хамгийн бага / их утга

Тархалт

<code>IQR()</code>	Квартилиудын хоорондох зай
<code>mad()</code>	Медианаас дундаж абсолют хазайлт
<code>sd()</code>	Стандарт хазайлт
<code>var()</code>	Дисперс

Хураангуй функцэд `na.rm = TRUE` өгч дутуу утгыг алгасна.

Мөрийн нэр

`Tidy` өгөгдөлд мөрийн нэр (`rownames`) хэрэглэдэггүй. Мөрийн нэрийг багана болгон хадгална (`tibble` багц):

`rownames_to_column(.data, var = "rowname")` Мөрийн нэрийг багана болгоно.

`a <- mtcars |> rownames_to_column(var = "C")`

`column_to_rownames(.data, var = "rowname")` Баганыг мөрийн нэр болгоно.

`has_rownames()`, `remove_rownames()` Шалгах, арилгах.

Хүснэгтүүдийг нэгтгэх

Хүснэгтийг зэрэгцүүлэх

`bind_cols(..., .name_repair)` Хүснэгтүүдийг хажуу тийш нь нийлүүлнэ. Мөрийн дараалал ижил эсэхийг заавал шалгаарай!

`bind_rows(..., .id = NULL)` Хүснэгтүүдийг доош нь нийлүүлнэ. `.id` – эх хүснэгтийг тэмдэглэх багана.

Мутацлах холболт (mutating joins)

`x` хүснэгтэд `y`-ээс тохирох баганыг нэмнэ.

```
x <- tibble(A = c("a", "b", "c"),
            B = c("t", "u", "v"), C = 1:3)
y <- tibble(A = c("a", "b", "d"),
            B = c("t", "u", "w"), D = 3:1)
```

`left_join(x, y, by = NULL, copy = FALSE, suffix = c(".x", ".y"), ..., keep = FALSE, na_matches = "na")` `x`-ийн бүх мөрийг үлдээж, `y`-ээс тохирохыг нэмнэ.

`right_join(x, y, ...)` `y`-ийн бүх мөрийг үлдээж, `x`-ээс тохирохыг нэмнэ.

`inner_join(x, y, ...)` Хоёуланд нь тохирсон мөрүүдийг л үлдээнэ.

`full_join(x, y, ...)` Хоёр хүснэгтийн бүх мөрийг үлдээнэ.

Холбох түлхүүрийг заах

`by = join_by(A)` Нийтлэг `A` баганаар холбоно.

`by = join_by(A, B)` Хэд хэдэн баганаар холбоно.

`by = join_by(A == C)` Өөр өөр нэртэй баганыг холбоно.

`by = join_by(A >= C)` Тэнцэлгүй нөхцөлөөр холбоно. Мөн `closest()`, `between()`, `overlaps()`, `within()`.

`suffix = c("1", "2")` Давхардсан баганын нэрт нэмэх дагавар.

`relationship = "one-to-one"` Холболтын төрлийг шалгана (`"many-to-one"` гэх мэт).

`multiple = "first", unmatched = "error"` Олон тохироо, тохироогүй мөрийг хэрхэн шийдэх.

Шүүх холболт (filtering joins)

`x`-ийн мөрүүдийг `y`-д тохирох эсэхээр нь шүүнэ, багана нэмэхгүй.

`semi_join(x, y, ...)` `y`-д тохирох мөртэй `x`-ийн мөрүүд.

`anti_join(x, y, ...)` `y`-д тохирох мөргүй `x`-ийн мөрүүд. Холболтын алдааг олоход хэрэгтэй.

Олонлогийн үйлдэл

`intersect(x, y)` `x` ба `y`-д хоёуланд нь буй мөрүүд.

`setdiff(x, y)` `x`-д байгаа боловч `y`-д байхгүй мөрүүд.

`union(x, y)` `x` эсвэл `y`-д буй мөрүүд (давхардалгүй). `union_all()` – давхардлыг хадгална.

`symdiff(x, y)` Зөвхөн аль нэгэнд нь байгаа мөрүүд.

`setequal(x, y)` Хоёр хүснэгт ижил мөрүүдтэй эсэх.

Nest холболт

`nest_join(x, y, ...)` `left_join` шиг боловч `y`-ийн тохирох мөрүүдийг жагсаалт-баганад хадгална.

Бүлэгтэй ажиллах нэмэлт функц

<code>n_groups(x)</code>	Бүлгийн тоо
<code>group_keys(x)</code>	Бүлэг бүрийн түлхүүр утгууд
<code>group_split(x)</code>	Бүлэг бүрийг тусдаа хүснэгт болгон жагсаалтад хуваана
<code>group_vars(x)</code>	Бүлэглэсэн хувьсагчдын нэр
<code>cur_group_id()</code>	<code>mutate()</code> / <code>summarise()</code> дотор бүлгийн дугаар
<code>n()</code>	Одоогийн бүлэг дэх мөрийн тоо

Бусад хэрэгтэй функц

`glimpse(x)` Хүснэгтийн баганауд, төрөл, эхний утгуудыг хэвтээгээр харуулна.

`reframe(.data, ...)` `summarise()` шиг боловч бүлэг бүрт дурын тооны мөр буцааж болно.

```
mtcars |> reframe(q = quantile(mpg, c(.25, .75)), .by = cyl)
```

`tibble(x = 1:3, y = x * 2)` Шинэ `tibble` үүсгэнэ; дараагийн багана өмнөхийг ашиглаж болно.

`as_tibble(df)` Дата фреймийг `tibble` болгоно.

Өгөгдлийн сантай ажиллах dbplyr

```
con <- DBI::dbConnect(RSQLite::SQLite(),
  ":memory:")
copy_to(con, mtcars)
db <- tbl(con, "mtcars")
db |> filter(cyl == 4) |> show_query()
db |> filter(cyl == 4) |> collect()
```

`dplyr` код автоматаар SQL болж өгөгдлийн сан дээр ажиллана. `show_query()` үүсгэсэн SQL-ийг харуулна, `collect()` үр дүнг R руу татна.